

DEV_IMPL_GUIDE - SUB-WF-ACTIVATE-01 Subscription Activation

1. Purpose

This guide explains activation implementation as the flow executes at runtime.

Use it with:

- DD_SUB-WF-ACTIVATE-01-Subscription_Activation-v1.0.md
- DD_SUB-WF-ACTIVATE-01-FLOW-WIK-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md
- DD_SUB-WF-FRAMEWORK-01-WORKERS-v1.0.md

The sequence below is the implementation path a developer should follow.

2. Runtime Actors

Runtime component	What it does
subscription-workflow-api pod	Receives activation API request, validates, starts operation.
subscription-workflow-worker pod/app	Handles SUB-WF framework topics and cross-cutting topics.
subscription-state-worker pod/app	Handles SUB-LM worker topics. May be consolidated with workflow worker in budget deployment.
billing-worker pod/app	Handles BIL-01 worker topics.
fulfillment-worker pod/app	Handles FUL/OSR worker topics.
Camunda	Stores and executes BPMN state.
Drools/KIE	Executes activation rules and returns decisions.
Kafka outbox dispatcher	Publishes outbox rows to Kafka.

pod/app means deployment can vary. A Laravel command, Java process, or container can handle multiple worker topics.

3. Before The Request Can Work

These records/artifacts must exist before testing activation.

3.1 Activation Config

Table:

subscription_activation_config

Required WIK row:

```
INSERT INTO subscription_activation_config (  
  operator_code,  
  cycle_anchor_overflow_default,  
  customer_self_service_enabled,  
  sales_field_activation_enabled,  
  scheduled_activation_enabled,  
  equipment_binding_validation_strict,  
  backdating_allowed,  
  backdating_max_days,  
  backdating_required_role,  
  updated_by  
)  
VALUES (  
  'WIK',  
  'PRORATE_PARTIAL_CYCLE',  
  FALSE,  
  TRUE,  
  TRUE,  
  TRUE,  
  TRUE,  
  7,  
  'SUBSCRIPTION_ADMIN',
```

```
'migration'
);
```

Meaning:

- WIK activation supports field sales and scheduled activation.
- Customer self-service activation is disabled.
- Equipment validation is strict.
- Backdating is allowed only for SUBSCRIPTION_ADMIN, up to 7 days.

3.2 Framework Operation Config

Table:

```
subscription_operation_config
```

Required WIK row:

```
INSERT INTO subscription_operation_config (
  operator_code,
  operation_kind,
  default_bpmn_process_key,
  operation_timeout_seconds,
  billing_call_timeout_seconds,
  fulfillment_call_timeout_seconds,
  feature_flags,
  enabled,
  updated_by
)
VALUES (
  'WIK',
  'ACTIVATE',
  'sub-activate-wik',
  900,
  60,
  120,
  '{}',
  TRUE,
  'migration'
);
```

Meaning:

- Activation for WIK starts Camunda process key sub-activate-wik.
- New activation operations are allowed because enabled = TRUE.

3.3 Drools Package

Required package:

```
rules.subscription.activate.wik
```

Required decisions:

```
[
  "validate-preconditions",
  "validate-backdating",
  "resolve-cycle-anchor-overflow",
  "validate-equipment-bindings"
]
```

3.4 Camunda BPMN

Required process definition:

```
process key: sub-activate-wik
```

The BPMN must contain service tasks using the worker topics listed in section 8.

4. Step 1 - Caller Sends Activation Request

Example caller:

```
FUL-02 onboarding flow
```

Endpoint:

```
POST /api/subscriptions/sub_01HZ_NEW/activate HTTP/1.1
Authorization: Bearer <service-or-user-jwt>
Idempotency-Key: activation:sub_01HZ_NEW:obr_01HZ_NEW_CUSTOMER
Content-Type: application/json
```

```
{
  "operatorCode": "WIK",
  "activationTrigger": "SALES_FIELD",
  "intendedActivationDate": "2026-05-18",
  "cycleAnchorOverflowPolicy": null,
  "equipmentBindings": [
    {
      "equipmentRequirement": "GPON_ONT",
      "equipmentRef": "ont_01HZ_NEW01"
    },
    {
      "equipmentRequirement": "ATA",
      "equipmentRef": "ata_01HZ_NEW01"
    }
  ],
  "initiatingWorkflowKind": "FUL_02_OBR_01",
  "initiatingWorkflowRef": "obr_01HZ_NEW_CUSTOMER"
}
```

Important:

- The request body does not decide actor role.
- The API must resolve actor/user/role from the JWT or service token.
- Idempotency-Key is mandatory.

5. Step 2 - Request Lands On subscription-workflow-api

The subscription-workflow-api pod receives the request.

It validates request shape first:

Field	Validation
operatorCode	Required.
activationTrigger	Required; allowed: BACKOFFICE_OPERATOR, CALL_CENTER, CUSTOMER_SELF_SERVICE, SALES_FIELD, SCHEDULED_ACTIVATION.
intendedActivationDate	Required valid date.
cycleAnchorOverflowPolicy	Optional; if present, allowed: ROUND_TO_28, ROUND_TO_1_NEXT_MONTH, PRORATE_PARTIAL_CYCLE.
equipmentBindings	Array; no duplicate equipmentRequirement.
equipmentBindings[].equipmentRequirement	Required.
equipmentBindings[].equipmentRef	Required.

If shape validation fails, return 400.

Example:

```
{
  "code": "VALIDATION_ERROR",
  "message": "equipmentBindings[0].equipmentRef is required."
}
```

6. Step 3 - API Resolves Auth

The API validates the token through FOUNDATION_AUTH.

Resolved auth context example:

```
{
  "actorUserId": "sales-agent-wik-042",
  "actorRole": "SALES_AGENT",
  "operatorCode": "WIK",
  "clientId": "ful-02-service"
}
```

Implementation rule:

Do not trust initiatingActorUserId or initiatingActorRole from the body.

7. Step 4 - API Loads Required Business Data

The API/service loads:

7.1 Activation Config

```
SELECT *
FROM subscription_activation_config
WHERE operator_code = 'WIK';
```

If missing:

```
{
  "code": "ACTIVATION_CONFIG_NOT_FOUND"
}
```

7.2 Framework Operation Config

```
SELECT *
FROM subscription_operation_config
WHERE operator_code = 'WIK'
AND operation_kind = 'ACTIVATE';
```

If disabled:

```
{
  "code": "OPERATION_KIND_DISABLED"
}
```

7.3 Subscription Summary

Load this from the SUB-LM source of truth, not from cache.

How to load it depends on deployment:

Deployment	How activation reads subscription
SUB-LM and SUB-WF in same application	Call the SUB-LM application service/read repository in-process, for example <code>SubscriptionReadService.getSummary(...)</code> .
SUB-LM and SUB-WF split into separate services	Call SUB-LM internal API, for example <code>GET /internal/subscriptions/{id}/summary</code> .

Do not let activation code issue random SQL against the subscription table unless that SQL is inside a SUB-LM-owned repository/service. Activation can consume the summary; SUB-LM owns how the row is read.

Do not use cache for this check. Activation needs authoritative current status because it is about to start a state-changing operation.

Example summary returned by SUB-LM:

```
{
  "subscriptionId": "sub_01HZ_NEW",
  "operatorCode": "WIK",
  "customerId": "cust_01HZ_NEW",
  "statusCode": "CREATED",
  "packageRef": "pkg_INET_VOIP_RES",
  "homepassId": "hp_01HZ_NRB",
  "billingMode": "PREPAID",
  "cycleModel": "CALENDAR_ANCHOR",
  "cycleFrequencyMonths": 1
}
```

7.4 Package Snapshot

Load this from the package/catalog module, not from SUB-LM.

Activation needs package and product/service composition because it must validate:

- Package exists.
- Package is ACTIVE.
- Package version to pin on activation.
- Required equipment for the package services.
- Minimum contract term, if applicable.

Runtime call:

```
GET /internal/packages/{packageRef}/activation-snapshot?operatorCode=WIK HTTP/1.1
Authorization: Bearer <subscription-workflow-service-token>
```

The target pod/module is the SIP/package-management API. If SIP delegates service composition to PLM-CFG, SIP can call PLM internally and return one activation snapshot. Activation should not call many catalog tables directly.

Who may call cache:

- Preferred: SIP/package-management owns its own cache and returns authoritative-enough activation snapshots through its API.
- Also acceptable: the subscription workflow service uses an approved `PackageCatalogClient` wrapper that implements read-through cache internally.
- Not allowed: activation business logic directly reads Redis/cache keys.

Read-through cache inside SIP or the approved client wrapper is allowed only as an optimization if:

- Cache entry includes package status/version/equipment requirements.
- Cache TTL is short.
- Cache miss falls back to SIP.
- If SIP is unavailable and no valid cache exists, activation fails safely.

Example response:

```
{
  "packageRef": "pkg_INET_VOIP_RES",
  "packageVersionId": "pkgver_003",
  "status": "ACTIVE",
  "requiredEquipment": ["GPON_ONT", "ATA"],
  "contractMinimumTermMonths": 12
}
```

7.5 HomePass Snapshot

Load this from the RLM/HomePass module, not from SUB-LM.

Activation needs HomePass data because it must validate that the selected HomePass can receive service.

Runtime call:

```
GET /internal/homepasses/{homepassId}/activation-snapshot?operatorCode=WIK HTTP/1.1
Authorization: Bearer <subscription-workflow-service-token>
```

The target pod/module is the RLM-CFG/HomePass API.

Who may call cache:

- Preferred: RLM/HomePass owns its own cache and returns authoritative-enough activation snapshots through its API.
- Also acceptable: the subscription workflow service uses an approved `HomePassClient` wrapper that implements read-through cache internally.
- Not allowed: activation business logic directly reads Redis/cache keys.

Read-through cache inside RLM or the approved client wrapper is allowed only as an optimization if:

- Cache entry includes sellability/current status.
- Cache TTL is short.
- Cache miss falls back to RLM.
- If RLM is unavailable and no valid cache exists, activation fails safely.

Example response:

```
{
  "homepassId": "hp_01HZ_NRB",
  "sellable": true,
  "technology": "GPON"
}
```

8. Step 5 - API Calls Drools

The API builds facts and calls:

rules.subscription.activate.wik

Facts sent:

```
{
  "operatorCode": "WIK",
  "operationKind": "ACTIVATE",
  "subscription": {
    "subscriptionId": "sub_01HZ_NEW",
    "statusCode": "CREATED",
    "customerId": "cust_01HZ_NEW",
    "packageRef": "pkg_INET_VOIP_RES",
    "homepassId": "hp_01HZ_NRB",
    "billingMode": "PREPAID",
    "cycleModel": "CALENDAR_ANCHOR",
    "cycleFrequencyMonths": 1
  },
  "package": {
    "packageRef": "pkg_INET_VOIP_RES",
    "packageVersionId": "pkgver_003",
    "status": "ACTIVE",
    "requiredEquipment": ["GPON_ONT", "ATA"],
    "contractMinimumTermMonths": 12
  },
  "homepass": {
    "homepassId": "hp_01HZ_NRB",
    "sellable": true,
    "technology": "GPON"
  },
  "request": {
    "activationTrigger": "SALES_FIELD",
    "intendedActivationDate": "2026-05-18",
    "cycleAnchorOverflowPolicy": null,
    "equipmentBindings": [
      {
        "equipmentRequirement": "GPON_ONT",
        "equipmentRef": "ont_01HZ_NEW01"
      },
      {
        "equipmentRequirement": "ATA",
        "equipmentRef": "ata_01HZ_NEW01"
      }
    ]
  },
  "actor": {
    "userId": "sales-agent-wik-042",
    "role": "SALES_AGENT"
  },
  "config": {
    "cycleAnchorOverflowDefault": "PRORATE_PARTIAL_CYCLE",
    "customerSelfServiceEnabled": false,
    "salesFieldActivationEnabled": true,
    "scheduledActivationEnabled": true,
    "equipmentBindingValidationStrict": true,
    "backdatingAllowed": true,
    "backdatingMaxDays": 7,
    "backdatingRequiredRole": "SUBSCRIPTION_ADMIN"
  },
  "runtime": {
    "today": "2026-05-18"
  }
}
```

Expected success decision:

```
{
  "eligible": true,
  "rejectionCode": null,
  "resolvedActivatedAt": "2026-05-18T10:00:00Z",
  "resolvedCycleAnchorDay": 18,
  "cycleAnchorOverflowApplied": null,
  "partialCycleDays": 0,
  "equipmentValid": true
}
```

If Drools returns rejection:

```
{
  "eligible": false,
  "rejectionCode": "ROLE_NOT_AUTHORIZED_FOR_TRIGGER",
  "rejectionDetail": "SALES_FIELD activation requires SALES_AGENT role."
}
```

```
}
```

the API returns:

```
403 Forbidden

{
  "code": "ROLE_NOT_AUTHORIZED_FOR_TRIGGER",
  "message": "SALES_FIELD activation requires SALES_AGENT role."
}
```

9. Drools Rules To Configure

The implementation should provide rules equivalent to the following.

9.1 Subscription Must Be CREATED

```
rule "activation requires subscription created"
when
  $facts : ActivationFacts(subscription.statusCode != "CREATED")
  $decision : ActivationDecision()
then
  if ($facts.subscription.statusCode == "ACTIVE") {
    $decision.reject("ALREADY_ACTIVE", "Subscription is already ACTIVE.");
  } else if ($facts.subscription.statusCode == "TERMINATED") {
    $decision.reject("SUBSCRIPTION_TERMINATED", "Terminated subscription cannot be activated.");
  } else {
    $decision.reject("INVALID_STATE_FOR_ACTIVATION", "Subscription must be CREATED before activation.");
  }
end
```

9.2 Package Must Be Active

```
rule "activation package must be active"
when
  $facts : ActivationFacts(packageInfo.status != "ACTIVE")
  $decision : ActivationDecision()
then
  $decision.reject("PACKAGE_RETIRED", "Package is not ACTIVE at activation time.");
end
```

9.3 HomePass Must Be Sellable

```
rule "activation homepass must be sellable"
when
  $facts : ActivationFacts(homepass.sellable == false)
  $decision : ActivationDecision()
then
  $decision.reject("HOMEPASS_NOT_SELLABLE", "HomePass is not sellable.");
end
```

9.4 Trigger Role Rules

```
rule "sales field activation role"
when
  $facts : ActivationFacts(
    request.activationTrigger == "SALES_FIELD",
    actor.role != "SALES_AGENT"
  )
  $decision : ActivationDecision()
then
  $decision.reject("ROLE_NOT_AUTHORIZED_FOR_TRIGGER", "SALES_FIELD activation requires SALES_AGENT role.");
end

rule "customer self service activation disabled"
when
  $facts : ActivationFacts(
    request.activationTrigger == "CUSTOMER_SELF_SERVICE",
    config.customerSelfServiceEnabled == false
  )
  $decision : ActivationDecision()
then
  $decision.reject("CUSTOMER_SELF_SERVICE_ACTIVATION_DISABLED", "Customer self-service activation is disabled for this o");
end
```

9.5 Cycle Anchor Overflow Rules

```
rule "calendar anchor overflow prorate partial cycle"
when
```

```

    $facts : ActivationFacts(
        subscription.cycleModel == "CALENDAR_ANCHOR",
        activationDayOfMonth in (29, 30, 31),
        effectiveOverflowPolicy == "PRORATE_PARTIAL_CYCLE"
    )
    $decision : ActivationDecision(rejected == false)
then
    $decision.setResolvedActivatedAt($facts.request.intendedActivationDate.atStartOfDay());
    $decision.setResolvedCycleAnchorDay(1);
    $decision.setCycleAnchorOverflowApplied("PRORATE_PARTIAL_CYCLE");
    $decision.setPartialCycleDays(daysUntilFirstOfNextMonth($facts));
end

rule "calendar anchor normal day"
when
    $facts : ActivationFacts(
        subscription.cycleModel == "CALENDAR_ANCHOR",
        activationDayOfMonth >= 1,
        activationDayOfMonth <= 28
    )
    $decision : ActivationDecision(rejected == false)
then
    $decision.setResolvedActivatedAt($facts.request.intendedActivationDate.atStartOfDay());
    $decision.setResolvedCycleAnchorDay(activationDayOfMonth($facts));
    $decision.setCycleAnchorOverflowApplied(null);
    $decision.setPartialCycleDays(0);
end

```

9.6 Equipment Rules

```

rule "activation required equipment must be supplied"
when
    $facts : ActivationFacts(missingRequiredEquipment not empty)
    $decision : ActivationDecision()
then
    $decision.reject("EQUIPMENT_MISSING", "Required equipment is missing: " + $facts.missingRequiredEquipment);
end

rule "activation strict equipment validation failed"
when
    $facts : ActivationFacts(
        config.equipmentBindingValidationStrict == true,
        invalidEquipmentRefs not empty
    )
    $decision : ActivationDecision()
then
    $decision.reject("EQUIPMENT_BINDING_INVALID", "One or more equipment references cannot be bound: " + $facts.invalidEquipmentRefs);
end

```

9.7 Backdating Rules

```

rule "activation backdating disabled"
when
    $facts : ActivationFacts(
        intendedActivationDateBeforeToday == true,
        config.backdatingAllowed == false
    )
    $decision : ActivationDecision()
then
    $decision.reject("BACKDATING_NOT_ALLOWED", "Backdated activation is disabled for this operator.");
end

rule "activation future date rejected"
when
    $facts : ActivationFacts(intendedActivationDateAfterToday == true)
    $decision : ActivationDecision()
then
    $decision.reject("FUTURE_ACTIVATION_DATE_NOT_ALLOWED", "Future activation must use scheduled activation.");
end

```

Drools must not call databases, APIs, Kafka, SUB-LM, BIL, FUL, or OSR. It receives facts and returns decisions only.

10. Step 6 - API Starts The Operation

After validation and Drools success, the API calls the SUB-WF framework start service.

Internal command:

```

{
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HZ_NEW",

```



```

"operationKind": "ACTIVATE",
"idempotencyKey": "activation:sub_01HZ_NEW:obr_01HZ_NEW_CUSTOMER",
"initiatingActorUserId": "sales-agent-wik-042",
"initiatingActorRole": "SALES_AGENT",
"initiatingWorkflowKind": "FUL_02_OBR_01",
"initiatingWorkflowRef": "obr_01HZ_NEW_CUSTOMER",
"bpmnVariables": {
  "activationTrigger": "SALES_FIELD",
  "intendedActivationDate": "2026-05-18",
  "resolvedActivatedAt": "2026-05-18T10:00:00Z",
  "resolvedCycleAnchorDay": 18,
  "cycleAnchorOverflowApplied": null,
  "partialCycleDays": 0,
  "equipmentBindings": [
    {
      "equipmentRequirement": "GPON_ONT",
      "equipmentRef": "ont_01HZ_NEW01"
    }
  ],
  "customerId": "cust_01HZ_NEW",
  "homepassId": "hp_01HZ_NRB",
  "packageRef": "pkg_INET_VOIP_RES",
  "packageVersionId": "pkgver_003",
  "billingMode": "PREPAID"
}
}

```

The framework:

1. Starts DB transaction.
2. Enforces idempotency.
3. Enforces in-flight operation uniqueness.
4. Inserts subscription_operation.
5. Inserts SubscriptionOperationStarted outbox row.
6. Commits.
7. Starts Camunda process sub-activate-wik.
8. Saves Camunda process instance ID.

11. Step 7 - API Returns Accepted

Response:

```

202 Accepted

{
  "operationId": "subop_01HZ_NEW_ACT",
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_NEW",
  "operationKind": "ACTIVATE",
  "bpmnProcessKey": "sub-activate-wik",
  "bpmnProcessInstanceId": "pi_01HZ_NEW_ACT",
  "currentState": "INITIATED",
  "finalState": null,
  "startedAt": "2026-05-18T10:00:00Z"
}

```

At this point the API request is done. The remaining work happens asynchronously in Camunda and workers.

12. Step 8 - Camunda Starts sub-activate-wik

Framework call to Camunda:

```

POST /engine-rest/process-definition/key/sub-activate-wik/start HTTP/1.1
Content-Type: application/json

{
  "businessKey": "subop_01HZ_NEW_ACT",
  "variables": {
    "operationId": {"value": "subop_01HZ_NEW_ACT", "type": "String"},
    "operatorCode": {"value": "WIK", "type": "String"},
    "subscriptionId": {"value": "sub_01HZ_NEW", "type": "String"},
    "operationKind": {"value": "ACTIVATE", "type": "String"},
    "activationTrigger": {"value": "SALES_FIELD", "type": "String"},

```

```

    "resolvedActivatedAt": {"value": "2026-05-18T10:00:00Z", "type": "String"},
    "resolvedCycleAnchorDay": {"value": 18, "type": "Integer"},
    "equipmentBindings": {
      "value": "[{"equipmentRequirement\\":\\"GPON_ONT\\",\\"equipmentRef\\":\\"ont_01HZ_NEW01\\"}]",
      "type": "Json"
    }
  }
}
}

```

13. Step 9 - BPMN Executes Worker Topics

In Camunda, each automated step is a service task. The service task declares a worker topic. A worker app subscribed to that topic executes it.

Order	BPMN service task does this	Worker topic	Consumed inputs	Produced outputs
1	Mark operation validating if modeled separately.	framework state update or local helper	operationId, currentState=VALIDATING	none
2	Put subscription in pending activation.	sub-lm-put-pending-status	subscriptionId, toStatus=PENDING_ACTIVATION, currentTransitionType=ACTIVATE, currentTransitionReasonCode=activationTrigger	priorStatusSnapshot, lmEventId
3	Compute cycle window.	sub-lm-compute-cycle-window	subscriptionId, referenceDate=resolvedActivatedAt	cycleWindowJson
4	Create pay-first charge.	bil01-charge-pre-authorize	subscriptionId, chargeCode=ACTIVATION_INTENT, chargeContext, cycleWindowJson	billableEventId, chargeCreatedAt
5	Wait for payment.	message catch payment-confirmed	billableEventId correlation	paidAt, paidAmount
6	Confirm charge.	bil01-confirm-charge	billableEventId	confirmedAt, chargeStatus
7	Activate service in network.	ful-call-activate	subscriptionId, activationCorrelationId=operationId + ':activate', equipmentBindings	activationFulfillmentId, activationTerminalState
8	Wait for fulfillment result.	message catch fulfillment-completed	activationFulfillmentId correlation	activationServices
9	Claim HomePass capacity.	ful-call-claim-homepasses	subscriptionId, homepassId, claimReason=SUBSCRIPTION_ACTIVATED	homepassClaimId
10	Bind equipment.	osr-equipment-bind	subscriptionId, equipmentBindings, bindCorrelationId=operationId + ':bind'	boundCount, boundInstanceRefs
11	Store activation master fields.	sub-lm-put-master-fields	subscriptionId, fieldsJson	lmEventId
12	Commit subscription active.	sub-lm-commit-state	subscriptionId, toStatus=ACTIVE, expectedFromStatus=PENDING_ACTIVATION	committedAt
13	Emit domain event.	kafka-emit	topic=sophix.subscription.subscription.activated, key=subscriptionId, payloadJson=SubscriptionActivated	outboxId
14	Send welcome notification.	notification-send	recipientKind=customer, recipientRef=customerId, notificationEventKind=SUBSCRIPTION_ACTIVATED	notificationId, dispatched
15	Finalize operation.	subscription-operation-finalize	operationId, finalState=COMPLETED	finalizedAt

This table is not a deployment list. One worker app may handle multiple topics.

14. Step 10 - Payment Message Arrives

BIL-01 emits or triggers a payment-confirmed correlation after customer pays.

Message to Camunda:

```
POST /engine-rest/message HTTP/1.1
Content-Type: application/json

{
  "messageName": "payment-confirmed",
  "correlationKeys": {
    "billableEventId": {
      "value": "bevt_01HZ_ACT_001",
      "type": "String"
    }
  },
  "processVariables": {
    "paidAt": {
      "value": "2026-05-18T10:03:00Z",
      "type": "String"
    },
    "paidAmount": {
      "value": 3850.00,
      "type": "Double"
    }
  }
}
```

If message arrives before BPMN waits:

1. Store unmatched message.
2. Retry correlation.
3. Alert if stale.

15. Step 11 - Fulfillment Message Arrives

FUL-03 completion is correlated to Camunda.

Success message:

```
POST /engine-rest/message HTTP/1.1
Content-Type: application/json

{
  "messageName": "fulfillment-completed",
  "correlationKeys": {
    "activationFulfillmentId": {
      "value": "ful_01HZ_ACT_001",
      "type": "String"
    }
  },
  "processVariables": {
    "activationTerminalState": {
      "value": "COMPLETED",
      "type": "String"
    },
    "activationServices": {
      "value": "{\"internet\":\"ACTIVE\",\"voice\":\"ACTIVE\"}",
      "type": "Json"
    }
  }
}
```

Failure message:

```
{
  "messageName": "fulfillment-failed",
  "correlationKeys": {
    "activationFulfillmentId": {
      "value": "ful_01HZ_ACT_001",
      "type": "String"
    }
  },
  "processVariables": {
    "failureReasonCode": {
      "value": "FULFILLMENT_FAILED",
      "type": "String"
    }
  },
}
```

```

        "failureReasonDetail": {
            "value": "OLT profile push failed.",
            "type": "String"
        }
    }
}

```

The failure message routes to compensation.

16. Step 12 - Success Event Is Emitted

After sub-lm-commit-state succeeds, the BPMN executes worker topic:

kafka-emit

Payload:

```

{
  "eventType": "SubscriptionActivated",
  "aggregateId": "sub_01HZ_NEW",
  "timestamp": "2026-05-18T10:00:18Z",
  "payload": {
    "operatorCode": "WIK",
    "subscriptionId": "sub_01HZ_NEW",
    "customerId": "cust_01HZ_NEW",
    "homepassId": "hp_01HZ_NRB",
    "operationId": "subop_01HZ_NEW_ACT",
    "activationTrigger": "SALES_FIELD",
    "packageRef": "pkg_INET_VOIP_RES",
    "packageVersionId": "pkgver_003",
    "activatedAt": "2026-05-18T10:00:18Z",
    "cycleModel": "CALENDAR_ANCHOR",
    "resolvedCycleAnchorDay": 18,
    "cycleAnchorOverflowApplied": null,
    "partialCycleDays": 0,
    "equipmentBindings": [
      {
        "equipmentRequirement": "GPON_ONT",
        "equipmentRef": "ont_01HZ_NEW01"
      }
    ],
    "billingMode": "PREPAID",
    "initiatingActorUserId": "sales-agent-wik-042",
    "initiatingActorRole": "SALES_AGENT"
  }
}

```

kafka-emit writes this to kafka_outbox. The outbox dispatcher publishes it later.

17. Compensation Flows

Compensation flows are BPMN branches. The names below are worker topics executed by Camunda service tasks.

17.1 Payment Timeout

Timer:

P7D

If payment does not arrive:

Order	BPMN executes worker topic	Inputs
1	bil01-revert-pending-charge	billableEventId, cancelReason=ACTIVATION_PAYMENT_TIMEOUT
2	sub-lm-revert-to-prior	subscriptionId, priorStatus=CREATED, failureReasonCode=ACTIVATION_PAYMENT_TIMEOUT
3	kafka-emit	topic=sophix.subscription.subscription.activationfailed, key=subscriptionId, payloadJson=SubscriptionActivationFailed
4	subscription-operation-finalize	operationId, finalState=FAILED, failureReasonCode=ACTIVATION_PAYMENT_TIMEOUT

17.2 Fulfillment Failed

Order	BPMN executes worker topic	Inputs
1	bil01-revert-pending-charge	billableEventId, cancelReason=FULFILLMENT_FAILED
2	sub-lm-revert-to-prior	subscriptionId, priorStatus=CREATED, failureReasonCode=FULFILLMENT_FAILED
3	kafka-emit	topic=sophix.subscription.subscription .activationfailed, key=subscriptionId, p ayloadJson=SubscriptionActivationFaile d
4	subscription-operation-finalize	operationId, finalState=FAILED, failureReasonCode=FULFILLMENT_FAILED

17.3 Equipment Bind Failed

Order	BPMN executes worker topic	Inputs
1	ful-call-terminate or approved rollback topic	subscriptionId, fulfillmentCorrelationId=operationId + ' :rollback-activation', reasonCategory=ACTIVATION_COMPENSATION
2	bil01-revert-pending-charge	billableEventId, cancelReason=EQUIPMENT_BINDING_INVALID
3	sub-lm-revert-to-prior	subscriptionId, priorStatus=CREATED, fai lureReasonCode=EQUIPMENT_BINDING_INVAL ID
4	kafka-emit	topic=sophix.subscription.subscription .activationfailed, key=subscriptionId, p ayloadJson=SubscriptionActivationFaile d
5	subscription-operation-finalize	operationId, finalState=FAILED, failureR easonCode=EQUIPMENT_BINDING_INVALID

17.4 SUB-LM Commit Failed

Do not mark completed.

Open Camunda user task:

```
manual-recovery-decision
```

Reason:

- Network may already be active.
- Billing may already be confirmed.
- A human must decide retry, force complete, or revert.

18. Worker App Consolidation

Recommended starting deployment:

```
subscription-state-worker:
- sub-lm-put-pending-status
- sub-lm-compute-cycle-window
- sub-lm-put-master-fields
- sub-lm-commit-state
- sub-lm-revert-to-prior
- subscription-operation-finalize
```

```
billing-worker:
- bil01-charge-pre-authorize
- bil01-confirm-charge
- bil01-revert-pending-charge
```

```
fulfillment-worker:
- ful-call-activate
- ful-call-claim-homepass
- osr-equipment-bind
- osr-equipment-unbind
```

```
cross-cutting-worker:
- drools-decide
```

- kafka-emit
- notification-send

Metrics must still be per topic:

```
worker_app
topic_name
operation_kind
operator_code
result
duration_ms
```

19. Manual Smoke Test

1. Create test subscription in CREATED.
2. Seed subscription_activation_config.
3. Seed subscription_operation_config.
4. Deploy Drools package rules.subscription.activate.wik.
5. Deploy Camunda BPMN sub-activate-wik.
6. Send activation API request.
7. Confirm subscription_operation row exists.
8. Confirm Camunda process instance exists.
9. Confirm subscription moves to PENDING_ACTIVATION.
10. Simulate payment-confirmed.
11. Simulate fulfillment-completed.
12. Confirm subscription becomes ACTIVE.
13. Confirm SubscriptionActivated outbox row exists.
14. Confirm operation final state is COMPLETED.

20. Common Mistakes

- Updating subscription status directly from activation API.
- Trusting actor role from request body.
- Starting Camunda before idempotency/concurrency reservation.
- Letting Drools call external APIs.
- Treating worker topic names as shell commands.
- Creating one worker deployment per topic by default.
- Publishing Kafka directly instead of writing outbox.
- Emitting SubscriptionActivated before SUB-LM commits ACTIVE.
- Dropping unmatched payment or fulfillment messages.